Практична робота №8

Безпека PHP-застосунків: пошук і усунення типових вразливостей

ІО-43 Череватов Артем

Варіант 20

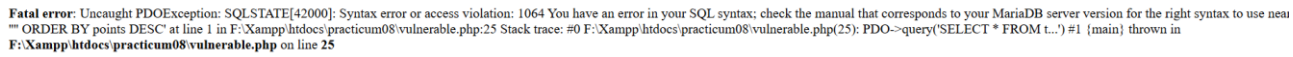
1. SQL-ін'єкція

Ціль перевірки: Параметр фільтрації списку команд за містом ($_GET['city'])

Тестовий payload:



Поведінка ДО виправлення:



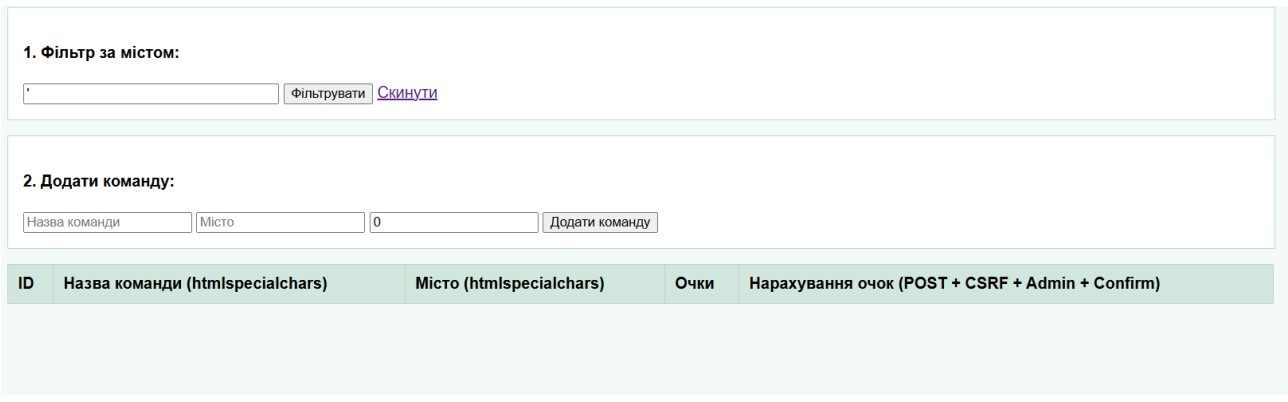
Код ДО виправлення:

```php
$cityFilter = $_GET['city'] ?? '';
if ($cityFilter !== '') {
    $sql = "SELECT * FROM teams WHERE city = '$cityFilter' ORDER BY points DESC";
    $teams = $pdo->query($sql)->fetchAll();
} else {
    $sql = "SELECT * FROM teams ORDER BY points DESC";
    $teams = $pdo->query($sql)->fetchAll();
}
?>
```

Код ПІСЛЯ виправлення:

```php
$cityFilter = trim($_GET['city'] ?? '');
try {
    if ($cityFilter !== '') {
        $stmt = $pdo->prepare('SELECT * FROM teams WHERE city = :city ORDER BY points DESC');
        $stmt->execute([':city' => $cityFilter]);
        $teams = $stmt->fetchAll();
    } else {
        $stmt = $pdo->query('SELECT * FROM teams ORDER BY points DESC');
        $teams = $stmt->fetchAll();
    }
```

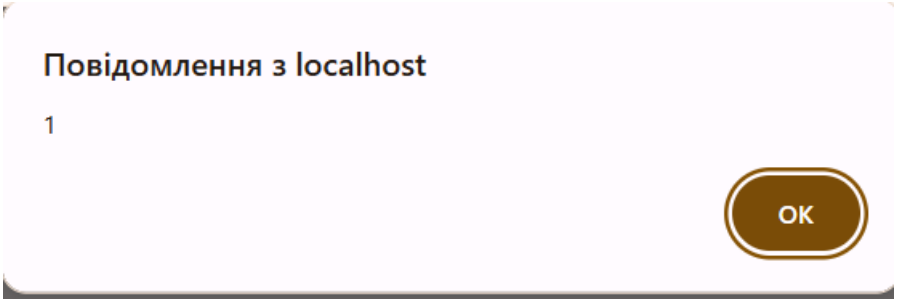
Поведінка ПІСЛЯ виправлення:



2. **Міжсайтовий скриптинг**
Ціль перевірки: Виведення назви команди (name) та міста (city) у HTML-таблиці турнірної таблиці.
Тестовий payload:



Поведінка ДО виправлення:



Код ДО виправлення:

```
<td><?= $row['name'] ?></td>
<td><?= $row['city'] ?></td>
```

Код ПІСЛЯ виправлення:

```
<td><?= htmlspecialchars($row['name'], ENT_QUOTES) ?></td>
<td><?= htmlspecialchars($row['city'], ENT_QUOTES) ?></td>
```

Поведінка ПІСЛЯ виправлення:

ID	Назва команди (htmlspecialchars)	Місто (htmlspecialchars)	Очки	Нарахування очок (POST + CSRF + Admin + Confirm)
2	"Шахтар"	"Донецьк"	41	3 Нарахувати очки Тест CSRF-атаки (403)
4	Полісся	Житомир	37	3 Нарахувати очки Тест CSRF-атаки (403)
3	Динамо	Київ	27	3 Нарахувати очки Тест CSRF-атаки (403)
6	Карпати	Львів	21	3 Нарахувати очки Тест CSRF-атаки (403)
5	Зоря	Луганськ	19	3 Нарахувати очки Тест CSRF-атаки (403)
8	<script>alert(1)</script>	Київ	0	3 Нарахувати очки Тест CSRF-атаки (403)

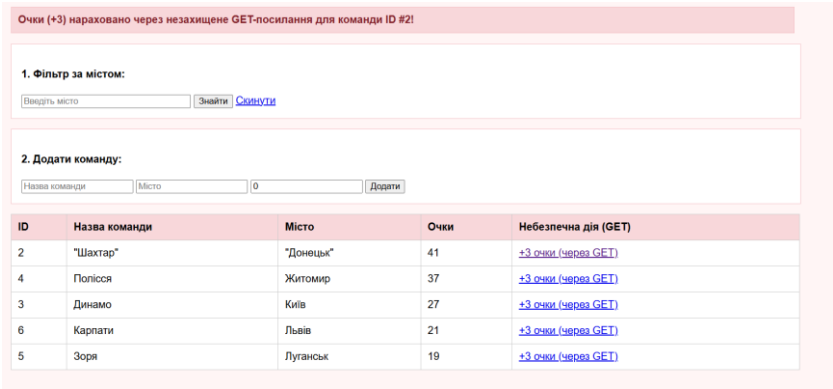
3. **Підробка міжсайтового запиту (CSRF) та незахищена дія — Форма нарахування очок**
Ціль перевірки: Нарахування турнірних очок команді
Тестовий payload / дія:

Відправка POST-запиту на зміну очок без токена або з підробленим токеном: action=updatePoints&id=1&points=3&csrf_token=wrong_token.

Спроба нарахувати очки переходом за прямим GET-посиланням ?action=updatePoints&id=1&points=3 без прав адміністратора або з від'ємним числом очок (points=-10)

Поведінка ДО виправлення:

Очки нараховувалися за звичайним GET-посиланням будь-яким користувачем без перевірки прав адміністратора та без перевірки CSRF-токена.



Очки (+3) нараховано через незахищене GET-посилання для команди ID #2!

1. Фільтр за містом:

Введіть місто | Знайти | Скинути

2. Додати команду:

Назва команди | Місто | 0 | Додати

ID	Назва команди	Місто	Очки	Небезпечна дія (GET)
2	"Шахтар"	"Донецьк"	41	+3 очки (через GET)
4	Полісся	Житомир	37	+3 очки (через GET)
3	Динамо	Київ	27	+3 очки (через GET)
6	Карпати	Львів	21	+3 очки (через GET)
5	Зоря	Луганськ	19	+3 очки (через GET)

Код ДО виправлення:

```php
if (isset($_GET['action']) && $_GET['action'] === 'updatePoints' && isset($_GET['id'])) {
    $id = $_GET['id'];
    $points = $_GET['points'] ?? 3;
    $pdo->query("UPDATE teams SET points = points + $points WHERE id = $id");
    $msg = "Очки (+3) нараховано через незахищене GET-посилання для команди ID #$id!";
}
```

Код ПІСЛЯ виправлення:

```php
if (empty($_SESSION['csrf_token'])) {
    $_SESSION['csrf_token'] = bin2hex(random_bytes(32));
}

if (isset($_GET['role'])) {
    $_SESSION['is_admin'] = ($_GET['role'] === 'admin');
}
$isAdmin = $_SESSION['is_admin'] ?? true;

$message = '';
$error = '';

if (isset($_GET['action']) && $_GET['action'] === 'updatePoints') {
    http_response_code(405);
    $error = 'Помилка 405 (Method Not Allowed): Нарахування очок через GET-параметр заборонено! Використовуйте
}
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    if (!hash_equals($_SESSION['csrf_token'] ?? '', $_POST['csrf_token'] ?? '')) {
        http_response_code(403);
        $error = 'Помилка 403 (Forbidden): Недійсний або відсутній CSRF-токен! Запит відхилено.';
    } else {
        $action = $_POST['action'] ?? '';

        try {
            if ($action === 'updatePoints') {
                if (!$isAdmin) {
                    http_response_code(403);
                    $error = 'Помилка 403 (Forbidden): Відмовлено в доступі! Нараховувати очки може лише адміні
                } else {
                    $id = filter_input(INPUT_POST, 'id', FILTER_VALIDATE_INT);
                    $points = filter_input(INPUT_POST, 'points', FILTER_VALIDATE_INT);

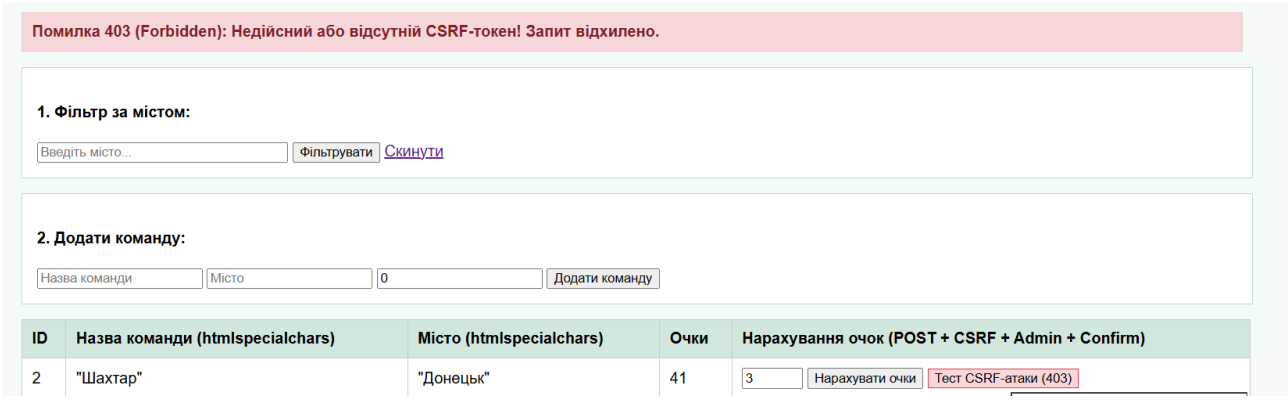
                    if (!$id || $points === false || $points <= 0 || $points > 100) {
                        http_response_code(400);
                        $error = 'Помилка 400 (Валідація): Очки мають бути додатним цілим числом від 1 до 100.'
                    } else {
                        $stmt = $pdo->prepare('UPDATE teams SET points = points + :points WHERE id = :id');
                        $stmt->execute([':points' => $points, ':id' => $id]);

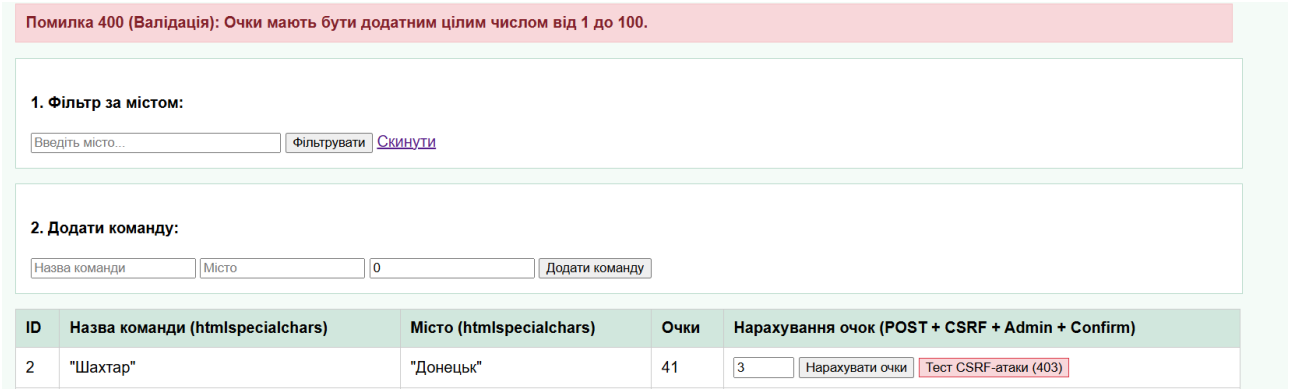
                        $cacheFile = sys_get_temp_dir() . '/cache_standings.json';
                        if (is_file($cacheFile)) unlink($cacheFile);

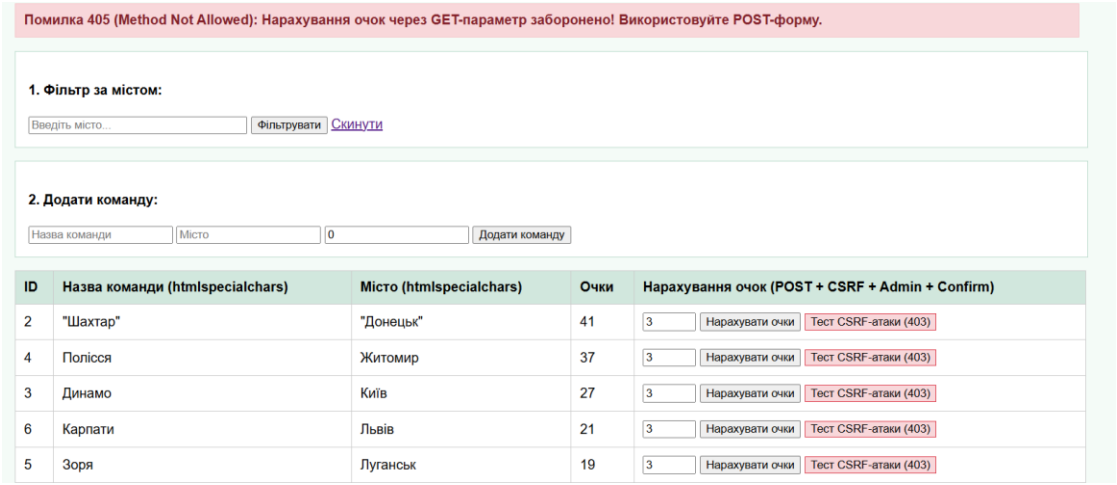
                        $message = "Успішно! Команді #$id нараховано +$points очок (перевірено CSRF та права ад
                    }
                }
            } elseif ($action === 'addTeam') {
                $name = trim($_POST['name'] ?? '');
                $city = trim($_POST['city'] ?? '');
                $points = filter_input(INPUT_POST, 'points', FILTER_VALIDATE_INT);
                if ($name === '' || $city === '' || mb_strlen($name) > 100 || mb_strlen($city) > 100) {
                    http_response_code(400);
                    $error = 'Помилка 400 (Валідація): Поля Назва та Місто не можуть бути порожніми.';
                } elseif ($points === false || $points < 0) {
                    http_response_code(400);
                    $error = 'Помилка 400 (Валідація): Початкові очки не можуть бути від\'ємними.';
                } else {
                    $stmt = $pdo->prepare('INSERT INTO teams (name, city, points) VALUES (:name, :city, :points
                    $stmt->execute([':name' => $name, ':city' => $city, ':points' => $points]);
                    $message = 'Нову команду успішно додано!';
                }
            }
        } catch (PDOException $e) {
            error_log("Database Error in index.php: " . $e->getMessage());
            http_response_code(500);
            $error = 'Внутрішня помилка сервера при збереженні даних.';
        }
    }
}
```

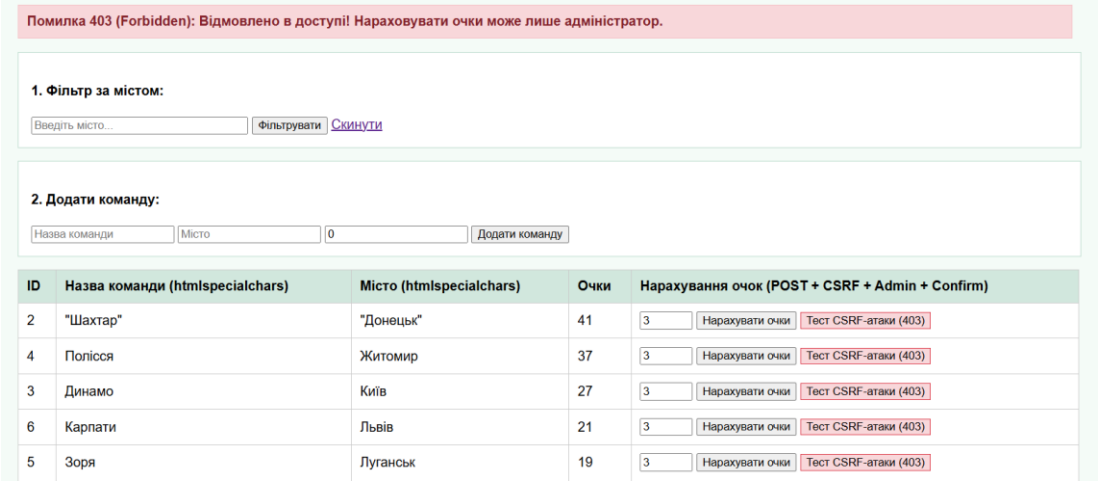
На форму нарахування очок також додано клієнтське підтвердження onsubmit="return confirm(...)" на JavaScript. Проте сам собою confirm() не є повноцінним захистом безпеки, оскільки виконується у браузері і легко обходиться прямим HTTP-запитом (наприклад, через curl), тому основний захист на сервері забезпечують метод POST, перевірка CSRF-токена та перевірка прав адміністратора.

Поведінка ПІСЛЯ виправлення:









Крок 6. Безпечна обробка помилок БД

```php
<?php
$dsn = 'mysql:host=localhost;dbname=practicum4;charset=utf8mb4';
$user = 'root';
$pass = '';

try {
    $pdo = new PDO($dsn, $user, $pass, [
        PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
        PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
    ]);
} catch (PDOException $e) {
    error_log("Database Connection Error: " . $e->getMessage());
    http_response_code(500);
    die('Внутрішня помилка сервера. Зверніться до адміністратора.');
}
?>
```